

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
УНИВЕРСИТЕТ ИТМО**

Дисциплина: Бэк-энд разработка

Отчет

**Домашнее задание “ДЗ5: Реализация межсервисного взаимодействия
посредством очередей сообщений”**

Выполнил:
Березина Софья
Группа
К3339

Проверил: Добряков Д. И.

Санкт-Петербург
2026 г.

Задача

Целью домашнего задания являлось подключение и настройка брокера сообщений RabbitMQ, а также реализация асинхронного межсервисного взаимодействия между микросервисами fitness-platform. Требовалось, чтобы сервисы не только обменивались HTTP-запросами, но и могли передавать события через очередь сообщений.

Ход работы

Для выполнения задания в docker-compose.yml был добавлен сервис rabbitmq на основе образа rabbitmq:3-management. Для RabbitMQ проброшены порты 5672 для AMQP-подключений и 15672 для web-интерфейса управления. Также настроен отдельный volume rabbitmq_data, чтобы данные брокера сохранялись между перезапусками контейнеров.

В качестве основной схемы обмена сообщениями используется topic exchange с именем fitness.events. Такой тип exchange позволяет маршрутизировать события по routing key и расширять систему без жесткой связности между отправителями и получателями. В рамках работы используются routing keys user.workout.assigned, user.workout.completed и blog.post.published.

Основной реализованный сценарий связан с завершением тренировки. Когда клиент вызывает endpoint POST /user-workouts/{id}/complete, сервис userworkouts обновляет статус назначенной тренировки в своей базе данных и публикует событие user.workout.completed в exchange fitness.events. В сообщении передаются eventId, время события, идентификаторы пользователя, тренировки и назначенной тренировки, а также метрики выполнения.

Сервис progress подключается к RabbitMQ, создает очередь progress.user_workout_completed и привязывает ее к exchange fitness.events по routing key user.workout.completed. После получения сообщения сервис разбирает payload и сохраняет запись в таблицу progress_records собственной базы данных. Для защиты от повторной обработки используется уникальный source_event_id.

Дополнительно сервис userworkouts публикует событие user.workout.assigned после назначения тренировки пользователю, а blog-service публикует событие blog.post.published после создания новой статьи. Эти события демонстрируют расширяемость событийной модели: к ним в

дальнейшем можно подключить уведомления, аналитику или историю активности без изменения producer-сервисов.

Вывод

В результате домашнего задания RabbitMQ был подключен к микросервисной версии fitness-platform, а межсервисное взаимодействие было дополнено асинхронными событиями. Ключевой сценарий завершения тренировки работает через брокер сообщений и приводит к изменению состояния другого сервиса: progress-service получает событие и сохраняет прогресс пользователя в собственной базе данных.